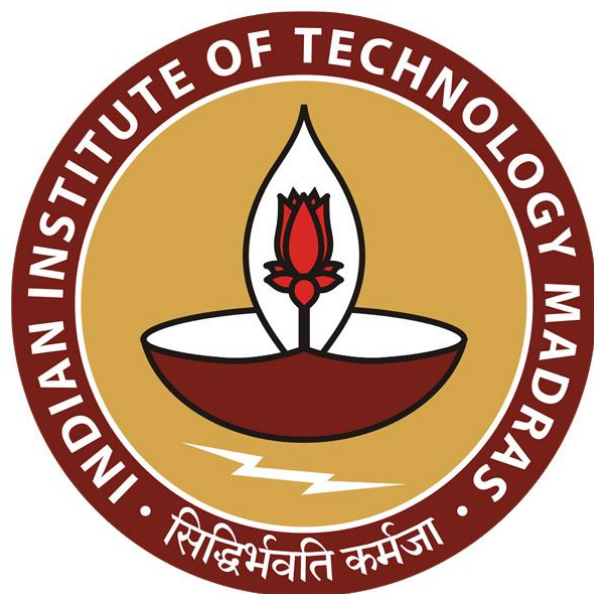


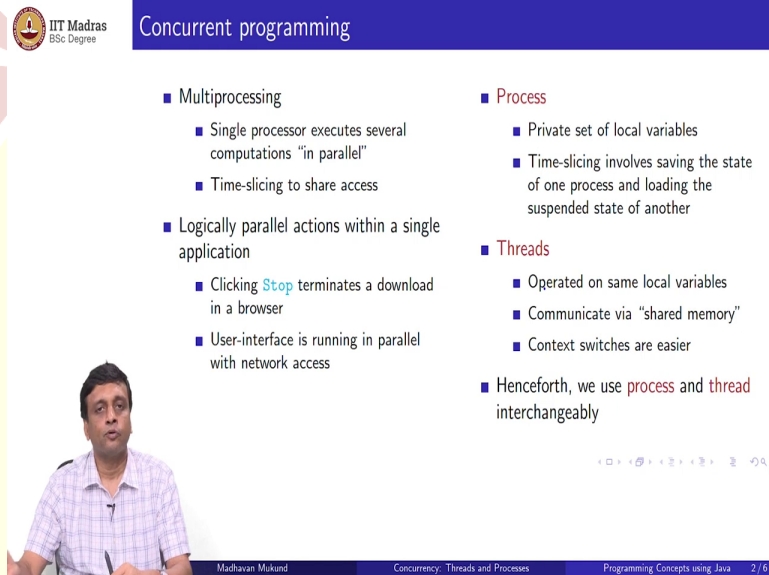


IIT Madras
ONLINE DEGREE

Programming Concepts Using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology Madras
Week: 10
Concurrency: Threads and Processes

So, let us look at a new topic, Concurrency.

(Refer Slide Time: 00:19)



The slide is titled "Concurrent programming" and features a list of bullet points under three main categories: Multiprocessing, Process, and Threads. A small video inset in the bottom left shows Professor Madhavan Mukund speaking. The slide is part of a presentation by IIT Madras BSc Degree.

- **Multiprocessing**
 - Single processor executes several computations "in parallel"
 - Time-slicing to share access
- **Logically parallel actions within a single application**
 - Clicking **Stop** terminates a download in a browser
 - User-interface is running in parallel with network access
- **Process**
 - Private set of local variables
 - Time-slicing involves saving the state of one process and loading the suspended state of another
- **Threads**
 - Operated on same local variables
 - Communicate via "shared memory"
 - Context switches are easier
- Henceforth, we use **process** and **thread** interchangeably

At the bottom of the slide, there is a navigation bar with the following text: "Madhavan Mukund", "Concurrency: Threads and Processes", "Programming Concepts using Java", and "2 / 6".

So, Concurrent programming involves running more than one thing at the same time. So, typically when we use a computer, we know that we can run many applications at the same time. Even, if we have only a single processor underneath. Of course, nowadays, most of our laptops have multiple cores or multiple CPUs.

But even with a single CPU, because the time it takes for actual computations to execute is very fast compared to the time it takes us to react and interact with our code, we get the illusion that multiple things are running at the same time. For instance, you might be running a browser in one window, you might be editing a document in another window, somewhere else another window might be checking your email, and so on.

So, these things seemingly happen in parallel. And actually, in practice, what is happening is the underlying operating system is scheduling these things by giving each task a slice of time. So, you have this thing called time slicing. So, you run one process or one task for a little while, then you run another one, then you run another one, you keep switching back and forth, and giving all of them some time to make progress so that the user overall has an illusion that all of these things are actually happening in parallel.

Now, even within one particular application, it sometimes makes sense to have things happening in parallel. So, we are really talking about different applications working in parallel, for instance, your browser and your mail client, and your document editor, and so on. Look at the browser itself. So, a browser is presumably a single application that you launch with a single click. But in this application, you can now open multiple tabs. So, you could be browsing different windows, so each window might be downloading information from a different website.

And, also when you then click on something to actually download a large file supposing you want to install some software and you are downloading that file, you can still continue to browse on another window and go ahead. Now even that download itself is something that can be terminated. So, you can at some point, decide that this download is taking too long. And you can click on a button and stop it.

So, all these interactions are happening within that one application, namely the browser. So, this browser itself has these multiple computations happening in parallel. And that is what we would like to talk about how to program such things. And what are the things that we need to look out for when we programmed done.

So, at the level of multiple different things happening, like a browser, and a text editor and something else, normally in an operating system, we have some abstract concept called a process. So, a process is an independent computation, it has its own local variables. And now when I switch from one process to another like whenever I switch to when the underlying operating systems, which is from the browser to the text editor, it will have to save all the values of the variables that the browser is using, and load the current state of the values that the text editor is using.

And then when it switches back, it will have to do the same thing in reverse. So, this requires this and saving and restoring the state of each process, each time we switch. However, when we are dealing within one application, then it may make sense for this state to be shared across all these processes these logical processes. And in such a situation, there is a different term which is used called a thread.

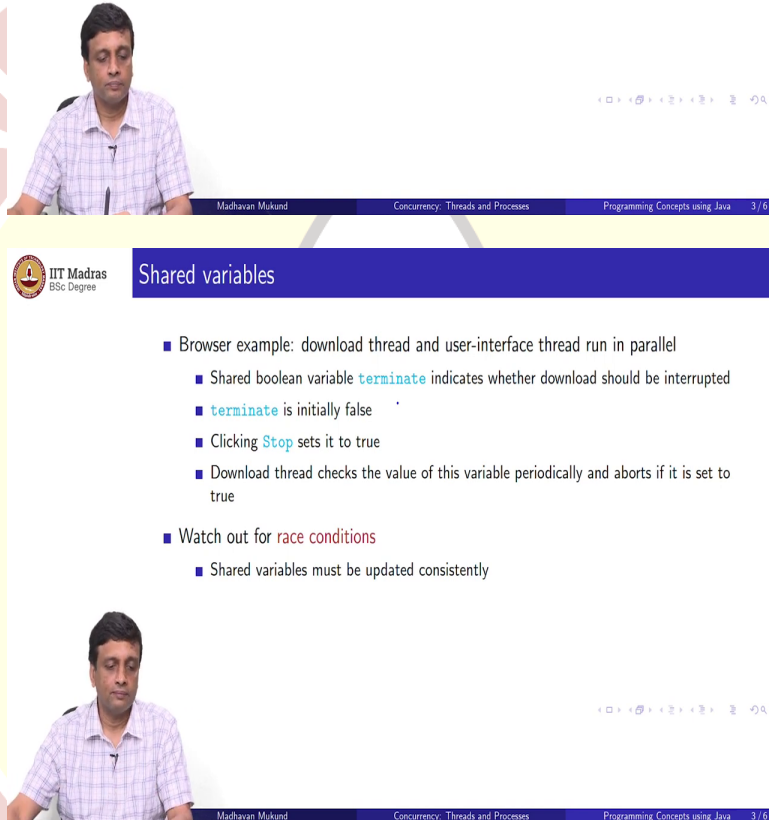
So, a thread is like a process except that different threads are usually running within a context where they shared the local variables. So, they have some shared variables. And they can therefore talk to each other also through the shared variables as we will explain. So, one advantage is that switching from one thread to another.

For example, if I am running a browser, and I switch tabs, then implicitly what is being displayed to me and what interacts with me is a different thread associated with the other tab. Now this switching is much faster if we do not have to unload and reload the state of the browser from the disk. So, therefore, threads are sometimes called lightweight processes.

So, we are not really going to distinguish between these two terms. So, we are generally going to talk about threads. But in some abstract discussion about how to coordinate threads and so on, we will use the term process occasionally, but we do not intend that there is any significant difference between the, use of the word processing the use of the word thread as far as we are concerned in this discussion of concurrent programming in Java.

(Refer Slide Time: 04:54)

- Browser example: download thread and user-interface thread run in parallel
 - Shared boolean variable `terminate` indicates whether download should be interrupted
 - `terminate` is initially false
 - Clicking `Stop` sets it to true
 - Download thread checks the value of this variable periodically and aborts if it is set to true



The screenshot shows a video lecture interface. At the top, there is a header with the IIT Madras logo and the text 'IIT Madras BSc Degree'. Below this is a blue bar with the title 'Shared variables'. The main content area is white and contains a list of bullet points. A presenter, a man in a light blue shirt, is visible in a small window at the bottom left of the slide. The slide content is as follows:

- Browser example: download thread and user-interface thread run in parallel
 - Shared boolean variable `terminate` indicates whether download should be interrupted
 - `terminate` is initially false
 - Clicking `Stop` sets it to true
 - Download thread checks the value of this variable periodically and aborts if it is set to true
- Watch out for **race conditions**
 - Shared variables must be updated consistently

At the bottom of the slide, there is a footer with the IIT Madras logo, the text 'IIT Madras BSc Degree', and the title 'Shared variables'. There is also a navigation bar with the text 'Concurrency, Threads and Processes' and 'Programming Concepts using Java 3 / 6'.

So, we said that threads can, share variables. So, this is one of the features that, distinguishes threads from processes. So, different concurrent threads are running. And they are usually under organized under a common application. So, they are doing some more or less the same thing. And they shared variables and the shared variables can actually be used to coordinate because different threads can read and write these variables and check the state of the other thread.

So, for instance, remember, we were talking about is browser, which could download something, but then we could accidentally or intentionally cancel this download if we felt it was taking too long. So, we could have the shared Boolean, variable called `terminate`, which indicates whether

or not that download should be terminated. And by default, when we start a download terminate is false.

At some point, when we click on the button to cancel a download, that is in a different thread, the download thread is happening in parallel, it is already downloading. But periodically it is checking the value of this variable terminate. Now in a different thread, where we are still interacting with a browser, we decide to cancel this download. What cancel does is it effectively sets that terminate variable from false to true.

So, at some point, in the middle of the download the periodically the terminate, the download thread, is checking the value of this variable. So, when it senses that the value is true, it will stop the download and cancel itself. So, in this way, the download thread and the user interaction thread of the browser interact through this particular example, in this variable called terminate.

So, what we will see is that this requires a little bit of care, because in our situation, we had one user who was pressing cancel, and we had one process which is checking this variable terminate. So, there is little scope for confusion. But in general, if we have many threads, and they are trying to update some variable at the same time, then we could have what is called a race condition.

So, imagine, for instance, that you are booking a ticket on a train, and there is one berth left. Now, of course, the railway reservation system is not restricted to one user at a time. So, everybody who logs in at the time and there is one berth left, we will see one berth left. And they may all start booking their ticket simultaneously concurrently. At some point, one of them will finish their payment and get the berth the other people will finish their payment and realize, but by now the berth is gone.

So, from the time you start booking the ticket to the time finish booking the ticket. The situation in terms of the train need not be the same if it is not programmed carefully. So, this is what we need to generally guarantee when we are doing concurrent programming, that the shared variables we are trying to update, which are being updated simultaneously by many different

components to many different threads, the shared variables are actually updated in a consistent way so that we do not accidentally corrupt the data that we have in the program.

(Refer Slide Time: 07:49)



Creating threads in Java

- Have a class extend `Thread`

```
public class Parallel extends Thread{  
    private int id;  
  
    public Parallel(int i){ id = i; }  
}
```

Madhavan Mukund

Concurrency: Threads and Processes

Programming Concepts using Java 4 / 6



Creating threads in Java

- Have a class extend `Thread`
- Define a function `run()` where execution can begin in parallel

```
public class Parallel extends Thread{  
    private int id;  
    public Parallel(int i){ id = i; }  
    public void run(){  
        for (int j = 0; j < 100; j++){  
            System.out.println("My id is "+id);  
            try{  
                sleep(1000); // Sleep for 1000 ms  
            }  
            catch (InterruptedException e){}  
        }  
    }  
}
```

Madhavan Mukund

Concurrency: Threads and Processes

Programming Concepts using Java 4 / 6

Creating threads in Java

- Have a class extend `Thread`
- Define a function `run()` where execution can begin in parallel

```
public class Parallel extends Thread{
    private int id;
    public Parallel(int i){ id = i; }
    public void run(){
        for (int j = 0; j < 100; j++){
            System.out.println("My id is "+id);
            try{
                sleep(1000); // Sleep for 1000 ms
            } catch (InterruptedException e){}
        }
    }
}
```



Madhavan Mukund

Concurrency: Threads and Processes

Programming Concepts using Java 4/6

Creating threads in Java

- Have a class extend `Thread`
- Define a function `run()` where execution can begin in parallel
- Invoking `p[i].start()` initiates `p[i].run()` in a separate thread

```
public class Parallel extends Thread{
    private int id;
    public Parallel(int i){ id = i; }
    public void run(){
        for (int j = 0; j < 100; j++){
            System.out.println("My id is "+id);
            try{
                sleep(1000); // Sleep for 1000 ms
            } catch (InterruptedException e){}
        }
    }
}

public class TestParallel {
    public static void main(String[] args){
        Parallel p[] = new Parallel[5];
        for (int i = 0; i < 5; i++){
            p[i] = new Parallel(i);
            p[i].start(); // Start p[i].run()
                        // in concurrent thread
        }
    }
}
```



Madhavan Mukund

Concurrency: Threads and Processes

Programming Concepts using Java 4/6

सिद्धिर्भवति कर्मजा

Creating threads in Java

- Have a class extend `Thread`
- Define a function `run()` where execution can begin in parallel
- Invoking `p[i].start()` initiates `p[i].run()` in a separate thread

```
public class Parallel extends Thread{
    private int id;
    public Parallel(int i){ id = i; }
    public void run(){
        for (int j = 0; j < 100; j++){
            System.out.println("My id is "+id);
            try{
                sleep(1000); // Sleep for 1000 ms
            }
            catch (InterruptedException e){}
        }
    }
}

public class TestParallel {
    public static void main(String[] args){
        Parallel p[] = new Parallel[5];
        for (int i = 0; i < 5; i++){
            p[i] = new Parallel(i);
            p[i].start(); // Start p[i].run()
                        // in concurrent thread
        }
    }
}
```



Madhavan Mukund

Concurrency: Threads and Processes

Programming Concepts using Java 4/6

Creating threads in Java

- Have a class extend `Thread`
- Define a function `run()` where execution can begin in parallel
- Invoking `p[i].start()` initiates `p[i].run()` in a separate thread
 - Directly calling `p[i].run()` does not execute in separate thread!

```
public class Parallel extends Thread{
    private int id;
    public Parallel(int i){ id = i; }
    public void run(){
        for (int j = 0; j < 100; j++){
            System.out.println("My id is "+id);
            try{
                sleep(1000); // Sleep for 1000 ms
            }
            catch (InterruptedException e){}
        }
    }
}

public class TestParallel {
    public static void main(String[] args){
        Parallel p[] = new Parallel[5];
        for (int i = 0; i < 5; i++){
            p[i] = new Parallel(i);
            p[i].start(); // Start p[i].run()
                        // in concurrent thread
        }
    }
}
```



Madhavan Mukund

Concurrency: Threads and Processes

Programming Concepts using Java 4/6

सिद्धिर्भवति कर्मजा

Creating threads in Java

- Have a class extend `Thread`
- Define a function `run()` where execution can begin in parallel
- Invoking `p[i].start()` initiates `p[i].run()` in a separate thread
 - Directly calling `p[i].run()` does not execute in separate thread!

```
public class Parallel extends Thread{
    private int id;
    public Parallel(int i){ id = i; }
    public void run(){
        for (int j = 0; j < 100; j++){
            System.out.println("My id is "+id);
            try{
                sleep(1000); // Sleep for 1000 ms
            }
            catch (InterruptedException e){}
        }
    }
}

public class TestParallel {
    public static void main(String[] args){
        Parallel p[] = new Parallel[5];
        for (int i = 0; i < 5; i++){
            p[i] = new Parallel(i);
            p[i].start(); // Start p[i].run()
                        // in concurrent thread
        }
    }
}
```



Madhavan Mukund

Concurrency: Threads and Processes

Programming Concepts using Java 4/6

Creating threads in Java

- Have a class extend `Thread`
- Define a function `run()` where execution can begin in parallel
- Invoking `p[i].start()` initiates `p[i].run()` in a separate thread
 - Directly calling `p[i].run()` does not execute in separate thread!
- `sleep(t)` suspends thread for `t` milliseconds
 - Static function — use `Thread.sleep()` if current class does not extend `Thread`
 - Throws `InterruptedException` — later

```
public class Parallel extends Thread{
    private int id;
    public Parallel(int i){ id = i; }
    public void run(){
        for (int j = 0; j < 100; j++){
            System.out.println("My id is "+id);
            try{
                sleep(1000); // Sleep for 1000 ms
            }
            catch (InterruptedException e){}
        }
    }
}

public class TestParallel {
    public static void main(String[] args){
        Parallel p[] = new Parallel[5];
        for (int i = 0; i < 5; i++){
            p[i] = new Parallel(i);
            p[i].start(); // Start p[i].run()
                        // in concurrent thread
        }
    }
}
```



Madhavan Mukund

Concurrency: Threads and Processes

Programming Concepts using Java 4/6

So, just to get the ball rolling, let us see how Java actually allows us to execute things in parallel. So, far, Java has been following a more or less conventional framework as far as programming goes. So, we call a function, if that calls another function, then this function will wait for that function to complete before it stops, starts again.

So, there is not any situation where two things are actually executing in parallel, they may be many things which are in principle running in parallel, but all except one, one kind of flow of execution are interrupted or halted, waiting for the next one to come back. Now in concurrent programming, what we are saying is that one thing can go on while something else is going on.

So, for this, Java has a class called Thread, which allows this kind of concurrent programming. So, if you want some functions to be executed in parallel, then the class that you have these functions in must extend thread. So, here, for instance, we are going to create a bunch of threads. And the only thing that we will know about these threads is some integer ID so that we can distinguish them from each other.

So, when you set up an object of this parallel class, the constructor just sets the id to be the value that you give it. Now, what happens when you actually run this? Well, what happens is that this is an interface sort of thing where thread guarantees that you have a function called run, which will actually be executed in parallel. So, in order to make this thread part work in parallel, there is a fixed signature as a function called run, which has a void return type. And this is what we will execute in parallel when we invoke this thread as we will see.

So, here for instance, what we are doing is we are 100 times. We are so each member that a thread was created by passing an ID. So, each thread, what it does is 100 times it prints its, name we will talk about the sleep in a second. So, each thread in parallel prints out its identity 100 times. Now to invoke this, we have to now create, of course, these objects of type parallel, and then start them off.

So, this is what we do, for instance, we could create 5 parallel threads in our main. For each of them, we assign this array to be a new parallel object with identity i. So, we have got an identity 0 1 2 3 4. And now the crucial thing is that we have to initiate these threads. And the way we do that is by calling this very special function called start. So, what start does notice that there is a discrepancy between start and, run.

So, run is the function, which is defined inside the class that extends thread start is the function that you call from outside on object of that class in order to invoke run. So, when I say start, it means execute the function run in parallel. So, it is important that you do not call run. So, run, of course, is a public function.

So, you can call run also from outside. But if you call run directly, it is like any other function it, will not happen concurrently, it will just run it one after the other sequentially as any other function would do. So, if you do this, for instance, then you have a loop, then it will do a few say

run instead of start here, it will do it for p[0], after p[0] finishes, that is, after p[0] finishes printing out its name, 100 times, then p[1] will start and so on, which is not what we want, we want p[0] to start.

And simultaneously, this will allow me to continue going. So, start is something which does not block as it were. So, it is a different kind of function, normally I call a function. I have to wait for that function to terminate before I can proceed. So, start is a different kind of function, it starts off this function, and continues with the current function in parallel. So, this main function here, this main loop, so this main loop, will not stop after p[0].

So, p[0].start() will kick off the first thread. Then you will come back and do p[1].start() while that thread is still running. So, in this sense, this loop will generate 5 concurrent computations. So, this sleep is a function which tells a thread to essentially go to sleep, so you provide a time, which is by in Java and milliseconds. So, this will say sleep.

So, 1000 milliseconds is 1 second, it a millisecond is 1 by 1000 of a second, like a millimeter is 1 by 1000 of a meter. So, 1000 milliseconds is 1 second. So, essentially, what we are telling this loop is print the ID of the current thread, wait 1 second, print the id of the current thread, again, wait 1 second, and so on.

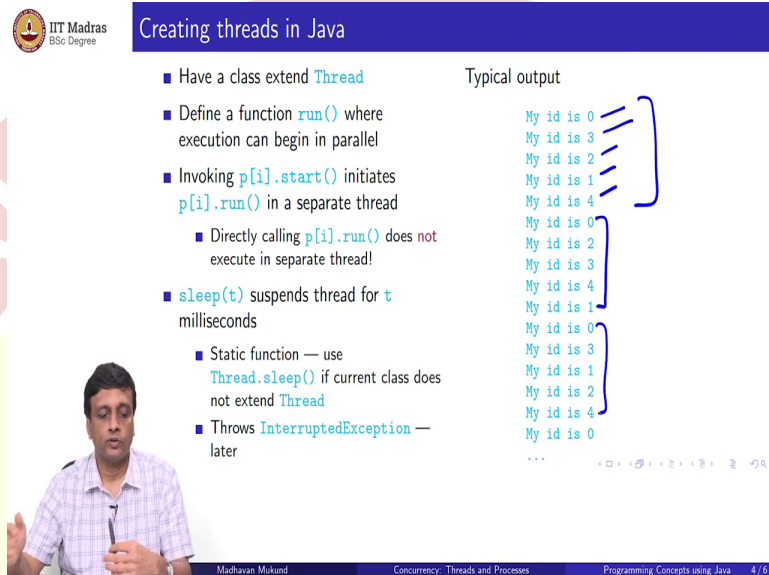
So, it will actually take 100 seconds for each thread or minimum of 100 seconds, because it will wait for 100 seconds in between each print statement plus the time to print. And all these 100 second computations are happening in parallel. So, if I do this for 5 threads, I should logically expect a long time to wait. I mean a few minutes to wait.

So, it is so turns out that this sleep actually calls an interrupted exception. So, when we call sleep, we must call this we must catch this interpreting section as well as Java will throw a type error and compiler error. And the other thing is that the sleep is a actually is a static function. So, here because we are extending thread, it is available to this class internally because it is inherited from thread.

But we will see that thread dot sleep is also a valid thing which you can call from outside because so sleep you can actually sleep any process you, any computation you want, it does not

really have to run in parallel, if you just want to wait some time before, supposing you want to print out numbers slowly on the screen between each `println`, you can actually sleep for some time and make your function wait.

(Refer Slide Time: 13:53)



Creating threads in Java

- Have a class extend `Thread`
- Define a function `run()` where execution can begin in parallel
- Invoking `p[i].start()` initiates `p[i].run()` in a separate thread
 - Directly calling `p[i].run()` does not execute in separate thread!
- `sleep(t)` suspends thread for `t` milliseconds
 - Static function — use `Thread.sleep()` if current class does not extend `Thread`
 - Throws `InterruptedException` — later

Typical output

```
My id is 0
My id is 3
My id is 2
My id is 1
My id is 4
My id is 0
My id is 2
My id is 3
My id is 4
My id is 1
My id is 0
My id is 3
My id is 1
My id is 2
My id is 4
My id is 0
```

Madhavan Mukund | Concurrency: Threads and Processes | Programming Concepts using Java | 4/6

So, if you run that code, this is what happens, we have started off these 5 threads in parallel, which are called 0 1 2 3 4. And each of them starts printing their identity with this 1 second gap. But because this is concurrent, and these are scheduled according to the underlying system, we really cannot control the order in which this happens.

So, we have a block here with the 0 thread prints its identity. So, this is a typical output each time you run this code, actually, you will probably get a different output. So, this one says that my ID is 0. So, `p[0]` has printed itself. And now normally you would think that because it is waiting for a while `p[1]` would be next but actually `p[3]` is next.

So, it has taken for whatever reason that I mean, we cannot really explain why this is the case, but we just get this kind of random behavior. So, these things are happening in parallel, and they are being scheduled in parallel and each of them may take a little bit of more or less time to do its work.

And therefore, the order in which these actions are executed is not predictable. So, here it saying, 0 print itself then 3, then 2, then 1, then 4. Now because of this 1 second delay, these for 5, actually happens in groups of 5. If we had a very small delay, then you might have even like two zeros happening before a 4 and so on.

So, we have different permutations here, there it was 0 3 2 1 4, the next time around, I get 0 2 4 3 1, then I get 0 1 2. And I might get the same permutation I cannot say, so the main thing, the main feature that you have, once you introduce concurrency is this kind of non determinism about the output, there is an interleaving.

This happens, this happens so, each concurrent computation is happening in parallel logically. And the way that happens is that each of them takes a step at a time and we cannot really predict at what sequence these steps are going to be scheduled and executed.

(Refer Slide Time: 15:43)

Java threads ...

- Cannot always extend `Thread`
 - Single inheritance
- Instead, implement `Runnable`

```
public class Parallel implements Runnable{  
    // only the line above has changed  
    private int id;  
    public Parallel(int i){ ... } // Constructor  
    public void run(){ ... }  
}
```

Madhavan Mukund | Concurrency: Threads and Processes | Programming Concepts using Java | 5/6

Creating threads in Java

- Have a class extend `Thread`
- Define a function `run()` where execution can begin in parallel
- Invoking `p[i].start()` initiates `p[i].run()` in a separate thread
 - Directly calling `p[i].run()` does not execute in separate thread!
- `sleep(t)` suspends thread for `t` milliseconds
 - Static function — use `Thread.sleep()` if current class does not extend `Thread`
 - Throws `InterruptedException` — later

```
public class Parallel extends Thread{
    private int id;
    public Parallel(int i){ id = i; }
    public void run(){
        for (int j = 0; j < 100; j++){
            System.out.println("My id is "+id);
            try{
                sleep(1000); // Sleep for 1000 ms
            } catch(InterruptedException e){}
        }
    }
}

public class TestParallel {
    public static void main(String[] args){
        Parallel p[] = new Parallel[5];
        for (int i = 0; i < 5; i++){
            p[i] = new Parallel(i);
            p[i].start(); // Start p[i].run()
                        // in concurrent thread
        }
    }
}
```



Madhavan Mukund

Concurrency: Threads and Processes

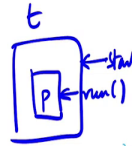
Programming Concepts using Java 4/6

Java threads ...

- Cannot always extend `Thread`
 - Single inheritance
- Instead, implement `Runnable`
- To use `Runnable` class, explicitly create a `Thread` and `start()` it

```
public class Parallel implements Runnable{
    // only the line above has changed
    private int id;
    public Parallel(int i){ ... } // Constructor
    public void run(){ ... }
}

public class TestParallel {
    public static void main(String[] args){
        Parallel p[] = new Parallel[5];
        Thread t[] = new Thread[5];
        for (int i = 0; i < 5; i++){
            p[i] = new Parallel(i);
            t[i] = new Thread(p[i]);
            // Make a thread t[i] from p[i]
            t[i].start(); // Start off p[i].run()
            // Note: t[i].start(),
            // not p[i].start()
        }
    }
}
```



Madhavan Mukund

Concurrency: Threads and Processes

Programming Concepts using Java 5/6

So, remember, of course, that Java has only single inheritance, and thread is a class. So, if we want to create an object, which can have a function that can run in parallel, and it extends thread, it cannot do anything else. So, fortunately, there is an interface which behaves the same as thread, but which allows us to do multiple things.

So, this is essentially a functional interface, like comparable, the only thing that Runnable guarantees is that there is this function run. So, as far as creating a class which can execute in parallel, instead of extending thread, I just have to say, implements Runnable. And then it can extend other things and implement other things also as usual.

So, this does not constrain it to being only an extend subclass of thread. So, other than that, everything else is the same. So, the previous definition that we had for so all this part is the same, everything below this is the same, it is only the header, which says that it implements Runnable, rather than extending thread.

On the other side, when we create these threads, now we have to do something slightly different in case we have objects, which implement Runnable, rather than extending thread. So, what we have to do is we have to actually create this object and create a thread from it. So, it is a two step process now.

So, earlier, if you remember, I had these thread objects, these things which extended thread, I just created 5 of them, and started them off. Now I have to have 5 thread objects, and 5 of these parallel objects which implement Runnable. And for each object, I have to kind of pass it. So, I have to create a thread object by passing this to the constructor.

So, here is what happens now. I have now two things here, I have an array of parallel objects, and I have an array of threads of the same size. So, for 0 to 4, what I do is I first create an object of this type, of the parallel type. Then I call this thread constructor by passing it this object which implements Runnable. And having done that, I can now start the thread. Now notice that I am starting the thread I am not starting the parallel object.

So, the so I start the thread, so there is a thread t, which contains an object P. So, when I say start here, what it does is it executes the run here. So, earlier, there was a same, I had a thread, which implemented run inside and from outside I press start, and then the runs has starts executing concurrently.

Here we are saying that this thread encapsulates something which actually has run. So, you press the start on the thread, because only thread has a start, Runnable does not have start, remember, Runnable has only run, so thread has many things, one of them is start and start implicitly calls run in parallel.

So, what we do is we press thread dot start, and it creates a run in parallel of the run, which is inside, so this is a sub so when we do Runnable instead of threads, we had the settle difference

that when we create the thread, we have to create the thread using the Runnable object as part of the constructor and then call the start on the thread itself.

(Refer Slide Time: 18:56)



Summary

- Common to have logically parallel actions with a single application
 - Download from one webpage while browsing another
- Threads are lightweight processes with shared variables that can run in parallel
- Use `Thread` class or `Runnable` interface to create parallel threads in Java



Madhavan Mukund

Concurrency, Threads and Processes

Programming Concepts using Java

6 / 6

So, to summarize, what we have said is that it is quite useful and important to have the capability to create these concurrent computations in our programming language. So, as an example, we talked about the web browser, which does many different things, even though it is a single application. And logically, these things all happen in parallel as far as the user is concerned, like downloading a file, or displaying a web page and so on.

And to implement this, there are these lighter weight processes called threads. So, a process is normally a separate application with its own variables and its own space, and it has to be unloaded and loaded each time. So, switching between one application and other application is generally a more expensive affair, then switching between one thread and another thread, where the context is more or less shared.

And this context the shared variables can be used to communicate like we saw that toy example where we use a terminate variable to tell the download thread whether it should cancel itself or not. And what we will see is that this sharing is really the critical part. I mean, just running things

in parallel, like that toy example that we gave of printing its IDs is not the typical case, you typically do not want 5 independent things to just run with no interaction until they complete.

So, that is really desirable only if you want to compute something like you want to compute. For example, if you want to take a matrix an array, and we want to add one to every element in the array, then you can of course, think of doing it in parallel for every position, because adding one to each position is independent of the others, there is no coordination required. But we are really thinking about situations where coordination is required.

So, we want to do things concurrently, in a sense, because we want to make things more flexible and efficient, it would be not very convenient. If the railway website shut down the moment one person logged it. And we have to wait for the thing to reopen after that person logs out. So, we would like to give multiple computations access to data. But we want to control the way that they do it.

So, we want a combination of this concurrency and control over consistency of the data. So, Java provides us with this thread class and this Runnable interface. So, we implement this function called run. And by using the function called start on the thread, this run will happen in parallel. So, this will allow us to actually run concurrent code within Java.